Material de preparación para la clase 1:

Integración de Clases, Objetos y Métodos

Algoritmos y Lógica de Programación - Primer Nivel B

Objetivo de la clase

Al finalizar la sesión, el estudiante será capaz de relacionar clases mediante atributos y parámetros, identificar la firma de un método y construir un programa sencillo en Java donde varios objetos colaboran entre sí.

1.2 Colaboración entre clases

En Java, una clase puede utilizar otra clase como parte de su estructura. Esto ocurre cuando una clase declara un atributo cuyo tipo es otra clase, o cuando recibe un objeto como parámetro en uno de sus métodos. Esta relación permite que los objetos se comuniquen y trabajen juntos para resolver un problema.

Por ejemplo, una clase Estudiante puede tener un atributo de tipo Carnet. De esta forma, cada estudiante posee su propio carnet, pero la información del carnet está organizada en una clase independiente.

```
public class Carnet {
    String codigo;
    String facultad;

    public Carnet(String codigo, String facultad) {
        this.codigo = codigo;
        this.facultad = facultad;
    }
}

public class Estudiante {
    String nombre;
    Carnet carnet; // Colaboración: un estudiante tiene un carnet

    public Estudiante(String nombre, Carnet carnet) {
        this.nombre = nombre;
        this.carnet = carnet;
    }

    public void mostrarInformacion() {
        System.out.println("Estudiante: " + nombre);
        System.out.println("Carnet: " + carnet.codigo);
```

```
        }
    }
```

La siguiente tabla resume las dos formas principales de colaboración:

Forma de colaboración	**Descripción**	**Ejemplo**
Como atributo	Una clase guarda un objeto de otra clase como característica.	Un Estudiante tiene un atributo de tipo Carnet.
Como parámetro	Un método recibe un objeto de otra clase para utilizar su información.	El método aplicar(Cuenta cuenta) recibe una cuenta.

1.3 Firmas de métodos

La firma de un método está formada por su nombre y la lista de parámetros que recibe. El tipo de retorno indica qué valor entrega el método al terminar su ejecución. Si un método no devuelve ningún valor, se declara con la palabra void.

Cuando un método devuelve un valor, debe utilizar la instrucción return. El tipo indicado en la firma debe coincidir con el tipo del valor devuelto.

Tipo de método	**Parámetros de entrada**	**Valor que devuelve**	**Ejemplo de firma**
void	Puede recibir o no recibir parámetros	No devuelve valor	public void saludar()
int	Puede recibir parámetros	Un número entero	public int obtenerEdad()
double	Puede recibir parámetros	Un número decimal	public double calcularPromedio(double nota1, double nota2)
String	Puede recibir parámetros	Texto	public String obtenerNombre()
boolean	Puede recibir parámetros	true o false	public boolean tieneSaldo()

Ejemplo: el método calcularArea de un rectángulo recibe dos valores decimales y devuelve un resultado de tipo double.

```
public double calcularArea(double base, double altura) {
    return base * altura;
}
```

1.4 Ejercicio guiado en vivo (VS Code)

En este ejercicio se modelará una cuenta bancaria y las transacciones realizadas sobre ella. La clase Cuenta almacena los datos básicos y permite consultar o modificar el saldo. La clase Transaccion recibe una cuenta como parámetro y aplica un depósito o un retiro.

Código de la clase Cuenta

```
public class Cuenta {
    String titular;
    double saldo;

    public Cuenta(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public double getSaldo() {
        return saldo;
    }

    public void depositar(double monto) {
        saldo = saldo + monto;
    }

    public void retirar(double monto) {
        if (monto <= saldo) {
            saldo = saldo - monto;
        } else {
            System.out.println("Fondos insuficientes.");
        }
    }

    public void mostrarSaldo() {
        System.out.println("Titular: " + titular);
        System.out.println("Saldo actual: " + saldo);
    }
}
```

Código de la clase Transaccion

```
public class Transaccion {
    String tipo;
    double monto;

    public Transaccion(String tipo, double monto) {
        this.tipo = tipo;
        this.monto = monto;
```

```
        }

        // La clase Transaccion recibe un objeto Cuenta como parámetro
        public void aplicar(Cuenta cuenta) {
            if (tipo.equals("deposito")) {
                cuenta.depositar(monto);
                System.out.println("Deposito realizado por: " + monto);
            } else if (tipo.equals("retiro")) {
                cuenta.retirar(monto);
                System.out.println("Retiro realizado por: " + monto);
            } else {
                System.out.println("Tipo de transaccion no valido.");
            }
        }
    }
```

Código de la clase principal

```
    public class Principal {
        public static void main(String[] args) {
            Cuenta cuenta1 = new Cuenta("Ana Torres", 500.0);

            cuenta1.mostrarSaldo();

            Transaccion deposito = new Transaccion("deposito", 250.0);
            deposito.aplicar(cuenta1);

            Transaccion retiro = new Transaccion("retiro", 100.0);
            retiro.aplicar(cuenta1);

            cuenta1.mostrarSaldo();
        }
    }
```

Durante el ejercicio, explique que el objeto cuenta1 se pasa al método aplicar de la clase Transaccion. La transacción no guarda una copia independiente del saldo: utiliza los métodos de la cuenta para modificarlo de forma controlada.

1.5 Cierre y solución de dudas

Para finalizar, se recomienda reforzar los siguientes puntos sobre la comunicación entre objetos:

- Una clase puede colaborar con otra cuando declara un atributo del tipo de esa clase o cuando recibe uno de sus objetos como parámetro.
- La firma de un método está formada por su nombre y sus parámetros; el tipo de retorno indica si el método devuelve un valor o no devuelve nada.

- Los objetos se comunican mediante sus métodos públicos. En el ejercicio, Transaccion utiliza los métodos de Cuenta para realizar depósitos y retiros sin acceder directamente al saldo.